实验二：使用 Verilog 设计 32 位 ALU　实验报告

姓名：杨清徽　　学号：2024201584　　2026 年 10 月

摘要　ALU 的加减法共用一个层次化超前进位加法器（4 位块 → 16 位段 → 32 位）。在 Vivado 2018.3 中，课程激励的 1,212,416 个向量全部通过：correct 全程为 1，也没有出现 X。在此基础上，本报告进一步回答一个问题：**“correct 恒为 1”到底证明了什么？** 先证明一点：激励的 ADD 段中，进位最多连续穿透 3 位，所以组间超前进位逻辑在 ADD 段里从未被检验过。然后用一个 Python 模型重放全部激励，预先算出 10 个典型错误（变异体）各自在第几 ns 第一次被抓到。xsim 实测结果与预测逐 ns 吻合，其中 2 个变异体能通过全部官方激励。最后补了一份五类向量的增强激励，杀死了全部 10 个变异体。

1　设计

1.1　加法器层次

表 1: 加法器层次。cla4、cla_unit4、cla16 沿用我在上一个实验（32 位超前进位加法器）中写的模块。原顶层的进位输入写死为 0，这里改写成带 cin/cout 的 cla32。

层次	模块	作用
4 位块	cla4	由 $p_i = a_i \oplus b_i$、$g_i = a_i b_i$ 和 cin 直接写出 $c_1..c_3$，并输出组传播 $P = \prod p_i$、组生成 G
16 位段	cla16 = 4×cla4 + cla_unit4	cla_unit4 用 4 个组的 P/G 一次性算出进入第 1–3 组的进位，并输出整段的 P/G
32 位	cla32 = 2×cla16	$c_{16} = G_{lo} + P_{lo} \cdot c_{in}$，$c_{out} = G_{hi} + P_{hi} \cdot c_{16}$



Elaborated Design 原理图，可以看到 cla32 → cla16 → cla4 / cla_unit4 的层次。

图 1: Elaborated Design 原理图，可以看到 cla32 → cla16 → cla4 / cla_unit4 的层次。

1.2　加减共用一个 CLA，以及溢出判断

```
wire        sub   = (Op == OP_SUB) || (Op == OP_SUBU);
wire [31:0] b_eff = B ^ {32{sub}};              // 减法: A + ~B + 1
cla32 u_cla(.a(A), .b(b_eff), .cin(sub), .sum(sum), .cout(cout));
wire ovf = (A[31] == b_eff[31]) && (sum[31] != A[31]);  // 只在 ADD/SUB 时输出
```

溢出式与 $c_{31} \oplus c_{32}$ 等价。 记最高位 $a = A_{31}$、$b = b_eff_{31}$、$c = c_{31}$，则 $s_{31} = a \oplus b \oplus c$，$c_{32} = ab + c(a \oplus b)$。

- 若 $a \neq b$：$c_{32} = c$，所以 $c_{31} \oplus c_{32} = 0$，与式中“同号”条件不成立的结果一致。
- 若 $a = b$：$s_{31} = c$，$c_{32} = a$，所以“$s_{31} \neq a$”恰好就是 $c_{31} \neq c_{32}$。

减法必须用“$\sim B$ + cin=1”一步算出，不能先求 $-B = \sim B + 1$ 再相加。 B=0x80000000 时，$-B$ 本身就溢出了，结果仍为 0x80000000，符号位是 1。用这个错误的符号去判溢出，会得出“0 − 0x80000000 不溢出”。这个错误对应 §3 的变异体 M4。

1.3　移位、逻辑运算与默认值

全部运算放在一个 always @(*) 的 case (Op) 里。块的开头先写 C = 0; Over = 0;，这样不会产生锁存器；对未定义的操作码，约定输出 C=0、Over=0。移位时被移的数是 B，移位量只取 A[4:0]。

SRA 单独写一个分支：C = $signed(B) >>> A[4:0];。如果把 SRL、SRA 合并成 Op[0] ? ($signed(B) >>> s) : (B >> s)，按 IEEE 1364 的表达式类型规则[1]，三目运算的另一支是无符号数，整个表达式就是无符号的。这个类型会向下传给 $signed(B)，于是 >>> 退化成逻辑右移。xsim 实测也是这样（§3 的 M6）。

2　官方激励仿真

时间轴。 激励共 1,212,416 个向量，每个向量保持 2 ns，段与段之间再隔 5 ns，$finish 在 **2,424,882 ns ≈ 2.42 ms**。实验指导中“波形图总长度约 2400ns”应为约 2400 µs。Vivado 默认只仿真 1000 ns，只能覆盖 ADD 段的前 500 个向量，所以工程里把 xsim.simulate.runtime 设成了 2500us。

表 2: 各段的起始时刻，用来定位波形。

段	ADD	SUB	ADDU	SUBU	SLL	SRL
起始 (ns)	0	524,293	1,048,586	1,081,359	1,114,132	1,376,281
段	**SRA**	**AND**	**OR**	**XOR**	**NOR**	**结束**
起始 (ns)	1,638,430	1,900,579	2,031,656	2,162,733	2,293,810	2,424,882

Tcl Console：$finish called at time : 2424882 ns。

图 2: Tcl Console：$finish called at time : 2424882 ns。

全程波形：op 逐段变化，correct 始终为 1。
图 3: 全程波形：op 逐段变化，correct 始终为 1。

在 correct 上用 Find Value 搜 0，结果为未找到。
图 4: 在 correct 上用 Find Value 搜 0，结果为未找到。

比“搜不到 0”更严格的检查。 `correct` 是用 == 算的，输出里有 X/Z 时它是 X 而不是 0，Find Value 搜 0 找不到 X。因此另写了 `sim/tb_monitor.v`。它例化了未改动的 `alu_tb`，用 !== 在每个向量稳定后检查 `correct`。批处理结果是：出错时长 0 ns，结束于 2,424,882 ns。为了确认这个监视器确实能报错，用两个临时设计测试过它：NOR 故意写成 NAND，报在 2,293,812 ns 首次出错；`Over` 不驱动，从 0 ns 起全程报错。

几个关键向量的放大波形如下。

ADD 段第一次溢出（402,302 ns）：0x08000000 + 0x78000000 = 0x80000000，Over=1。
图 5: ADD 段第一次溢出（402,302 ns）：0x08000000 + 0x78000000 = 0x80000000，Over=1。

SUB 段第一个向量 0 − 0（524,293 ns）：这一次进位要穿透全部 32 位，结果为 0。
图 6: SUB 段第一个向量 0 − 0（524,293 ns）：这一次进位要穿透全部 32 位，结果为 0。

SUB 段第一次溢出（525,205 ns）：0 − 0x80000000，Over=1。
图 7: SUB 段第一次溢出（525,205 ns）：0 − 0x80000000，Over=1。

SRA 补 1（1,653,150 ns）：B=0x80000000，A=1，C=0xC0000000。
图 8: SRA 补 1（1,653,150 ns）：B=0x80000000，A=1，C=0xC0000000。

3 “correct 恒为 1”证明了什么

3.1 激励的结构性盲区

ADD 段的操作数是 ii<<i 和 jj<<j，其中 ii、jj ⩽ 15，所以两个数的非零位各自落在一个 4 位窗口里。

设第 m 位产生进位（$a_m = b_m = 1$）。两个窗口都包含第 m 位，所以都落在 $[m-3,\, m+3]$ 内。进位只能在恰好一个操作数为 1 的位上继续传，并且不会超出这个范围，所以**最多连续穿透 3 位**。

组间项 $P_k \cdot C_k$ 要求一个进位穿透整组 4 位，所以在 ADD 段里不可能出现。16 位那一层的 $P_{lo} \cdot c_{in}$ 也一样，因为 ADD 时 $c_{in} = 0$。ADDU 段的窗口只有 2 位，最多穿透 1 位。

SUB 则完全不同：$a - b = a + \sim b + 1$，而 $\sim b$ 几乎全是 1。第一个向量 0 − 0 实际算的是 0 + 0xFFFFFFFF + 1，进位穿透全部 32 位。

表 3: 由 `scripts/tb_model.py` 逐向量统计，与上面的推导完全一致。另外，三个移位段中 A[31:5] 始终为 0，激励里也没有出现未定义的操作码。

段	向量数	组间 P_kC_k 触发	最长穿透	穿透 ⩾4	⩾16	=32
ADD	262,144	**0**	**3**	0	0	0
SUB	262,144	250,642	32	251,699	130,652	3,266
ADDU	16,384	**0**	**1**	0	0	0
SUBU	16,384	15,684	32	15,729	10,172	1,250

3.2 变异测试：先预测，再实测

我写了 10 个变异体，每个只改动一两行，模拟一类典型错误。

预测。 `scripts/tb_model.py` 按 `src/alu.v` 的层次逐位实现了 CLA，可以打开各个变异开关，并逐个向量重放官方激励。由此预测每个变异体的首次出错时刻和累计出错时长。预测结果**先单独提交到 git**（提交 349cca5），然后才运行任何变异体的仿真。

实测。 用 xsim 跑“官方激励 + 监视器”，不中途停止。

表 4: 10 个变异体的预测与 xsim 实测，两项指标全部精确一致。

ID	模拟的错误	预测首次出错 (ns)	xsim 实测	出错时长 预测 / 实测 (ns)
M0	沿用旧加法器写死的 c0=0，SUB 没接 cin	524,293	524,293	557,066 / 557,066
M1	`cla_unit4` 的 c_1 漏掉 $P_0 \cdot c_{in}$	524,293	524,293	498,564 / 498,564
M1′	同 M1，但 ADD/ADDU 单独用一套 CLA	**存活**	存活	0 / 0

ID	模拟的错误	预测首次出错 (ns)	xsim 实测	出错时长 预测 / 实测 (ns)
M2	顶层 c_{16} 漏掉 $P_{lo} \cdot c_{in}$	524,293	524,293	175,122 / 175,122
M3	cla4 的 c_1 漏掉 $p_0 \cdot c_{in}$（组内）	2,110	2,110	564,150 / 564,150
M4	先求 $\sim B + 1$，再用它的符号判溢出	525,205	525,205	15,365 / 15,365
M5	移位量用整个 A	**存活**	存活	0 / 0
M6	SRL/SRA 合进一个三目表达式	1,653,150	1,653,150	27,781 / 27,781
M7	ADDU/SUBU 也输出溢出	1,078,008	1,078,008	1,363 / 1,363
M8	把进位输出当作溢出	402,302	402,302	270,397 / 270,397

结果说明三件事。

第一，组间超前进位是否正确，官方激励只能靠 SUB 段来检验。 M0、M1、M2 的错误都在组间进位，它们能通过 ADD 段全部 262,144 个向量，直到 SUB 段的第一个向量 0 − 0 才被抓到。例如 M1 在这个向量上输出 0x00F000F0：两个 16 位段的第 1 块都收不到进位。反过来，**只要加法和减法不共用同一个 CLA（M1′），坏掉的组间超前进位逻辑就能通过全部官方激励**。M5 则利用了“A 只取 0..31”这个盲区。

第二，这些错误被发现得很晚，而且出错的向量很少。 M8 把进位当作溢出，在 ADD 段跑完 77% 后才第一次出错。M7 在 ADDU 段跑完 90% 后才出错，全程只有 679 个向量出错，在 2.42 ms 的波形上只占 1,363 ns。只看全局波形，很容易把这样的错误漏掉。

第三，模型和仿真器在每一个 ns 上都一致。 这说明 §3.1 的分析和 §1.3 对符号性规则的理解（M6）都是对的。

3.3 增强激励

`sim/alu_tb_plus.v` 是自检式激励：用 `!==` 比较 {Over, C}，按向量类别分别统计错误数。共 226,744 个向量，结果如下。

表 5: 增强激励的结果。五类向量分别是：边界值 {0, 1, 2, 0x7FFFFFFF, 0x80000000, 0x80000001, 0xFFFFFFFE, 0xFFFFFFFF, 0x55555555, 0xAAAAAAAA} 的全组合 × 11 种运算；长进位链 $(2^n - 1) + 1$、$x+ \sim x$、$x + (-x)$、$x - x$；A[31:5] 随机非零的移位；每种运算 2 万组固定种子的随机数；53 个未定义操作码。

设计	总错	边界值	长进位链	移位量高位	随机	未定义 Op
正确设计	**0**	0	0	0	0	0
M0	41,532	200	1,332	0	40,000	0
M1	5,884	80	939	0	4,865	0
M1′	1,655	28	426	0	1,201	0
M2	437	32	404	0	**1**	0
M3	71,490	162	1,376	0	69,952	0
M4	10	**10**	0	0	**0**	0
M5	59,449	128	0	1,842	57,479	0
M6	10,002	40	0	270	9,692	0
M7	10,392	46	315	0	10,031	0
M8	21,013	94	785	0	20,134	0

10 个变异体全部被杀死，而且**每一类向量都有别的类替代不了的作用**：

- **M4 只被边界值类抓到。** 随机数取到 B=0x80000000 的概率是 2^{-32}，所以 8 万个随机加减向量一次也没碰上。
- **对 M2，随机类在 8 万个向量里只命中 1 次。** M2 只在 SUB 且 A[15:0]=B[15:0] 时出错，概率是 2^{-16}；随机类有 4 万个 SUB/SUBU 向量，期望命中约 0.6 次。长进位链类则命中了 404 次。
- **M5 主要靠移位量高位类和随机类抓到。** 官方激励中 A[31:5] 始终为 0，所以抓不到它。

由此得到两个结论：要可靠地检验超前进位逻辑，需要专门构造长传播链的向量；要可靠地检验溢出判断，需要边界值向量。这两类向量都不能指望靠随机数碰上。

4 AI 使用声明与参考资料

AI 使用声明。 本实验使用了 Claude（Anthropic 公司，通过 Claude Code 使用 Claude Opus 5.5 模型），用于以下工作：

- 分析课程激励；
- 在我之前写的加法器模块基础上组装 ALU；
- 编写批处理仿真、监视器、Python 激励模型、变异体和增强激励的脚本，并运行这些实验；
- 起草本报告。

延伸方向（激励强度与变异测试）由我本人选定。Vivado GUI 中的截图由我本人完成。所有数据都可以用仓库中的脚本和 `results/` 复现，并已由我本人核对。

参考资料：

- [1] IEEE Std 1364-2005, IEEE Standard for Verilog Hardware Description Language, §5.5 Signed expressions;
- [2] AMD/Xilinx, Vivado Design Suite User Guide: Logic Simulation (UG900), v2018.3;

· [3] D. A. Patterson, J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, MIPS Edition, 第 3 章（溢出检测）与附录（超前进位）。

附录　复现方法

所有命令都在 `lab2/` 下运行，需要 Windows 上的 Vivado 2018.3。WSL 通过 `cmd.exe` 调用 xvlog/xelab/xsim，源码会先复制到 `E:\vivado_proj\lab2_alu\batch\`。

```
python3 scripts/xsim.py official src/alu.v          # 官方激励 + 监视器
python3 scripts/tb_model.py                          # 覆盖统计与变异体预测
python3 scripts/mutate.py                            # 变异体：官方激励，与预测比对
python3 scripts/mutate.py --plus                     # 正确设计与变异体：增强激励
```

GUI 工程由 `scripts/make_project.tcl` 生成，位于 `E:\vivado_proj\lab2_alu\lab2_alu.xpr`，只包含交付的 `src/alu.v` 和课程下发的 `alu_tb.v`。命令与原始数据的完整记录见 `实验日志.md` 和 `results/`。